LW-Driven Bubble-Free Five-Stage RISC-V Processor: Implementation and FPGA Verification

Nov 3, 2026 · @Loka Veera Sai Teja

Loka Veera Sesha Sai Teja Gantasala, B.Tech Electronics and Communication Engineering, IIIT Sri City

Abstract

In a classic five-stage pipeline, an instruction that uses the result of a load immediately has to wait one cycle, because the loaded value is only available at the end of the MEM stage. Kumar and Ray (IEEE TCAS-I, January 2026) proposed an LW-driven bubble mitigation scheme that computes the load address one stage earlier and removes most of these stalls without changing the clock period.

For this project I built two RV32I cores on the same base code: a conventional core that stalls on every load-use hazard, and a core with the paper's address generator and bubble mitigation control unit. Both were verified in simulation and then run on an Artix-7 FPGA (XC7A35T).

On a 10-load test program the bubble-free core removed 8 of 10 bubbles and finished in 34 cycles instead of 41, a 17.1% reduction, with identical final register values. Across a 45-program sweep modelled on the paper's Table III, the reduction was 18.27%, close to the 19.37% reported in the paper. The same 41-versus-34 result was observed on the physical board.

In hardware the bubble-free core needed 12.2% more LUTs and registers and ran at a 5.2% lower maximum frequency, while its estimated dynamic power fell from 22 to 19 mW. Both changes are in line with the paper's Table II.

Results at a glance

Metric	Baseline (stall)	LW-driven (bubble-free)	Change
Bubbles on the 10-load program	10	2	-80%
Cycles, 10-load program, simulation	41	34	-17.1%
Cycles, 10-load program, on the FPGA	41	34	-17.1%
Total cycles, 45-program Table III-style sweep	591	483	-18.3%
Paper's Table I program, last result written back	cycle 11	cycle 10	-1 cycle
Final register state (x28, x29, x30)	0, 111, 112	0, 111, 112	identical
LUTs + registers	2076	2330	+12.2%
Max frequency (MHz)	160.8	152.5	-5.2%
Dynamic power (mW, estimated)	22	19	-13.6%
Dynamic energy for the 10-load program (nJ)	9.0	6.5	-28.4%

1. Introduction

Loads are among the most common instructions in embedded code, and very often the next instruction needs the loaded value straight away. A standard five-stage pipeline handles this by freezing the front of the pipeline for one cycle and inserting a bubble. Each bubble is a wasted cycle, and in load-heavy code they add up.

The paper by Kumar and Ray targets exactly this case for low-end, energy-constrained cores. Instead of adding a deeper pipeline or a cache trick, it moves the load's address calculation from EX into ID. The memory read can then start one stage earlier, and the data is ready in time for the dependent instruction.

The objectives of this project were:

1. Build a working baseline RV32I five-stage pipeline with forwarding and a conventional load-use stall.

2. Add the paper's address generator and bubble mitigation control unit to a copy of it, covering all three hazard cases.
3. Show, in simulation, when a bubble is detected and whether it is removed.
4. Measure the cycle saving on test programs similar to the paper's Table III.
5. Run both cores on a real FPGA and confirm the same result in hardware.

2. Background: why a load causes a bubble

The pipeline has five stages: instruction fetch (IF), decode and register read (ID), execute (EX), memory access (MEM) and write-back (WB). Most data hazards are solved by forwarding a result from the EX/MEM or MEM/WB register back into the ALU inputs.

A load is different. Take a load, lw x3, 0(x6), followed straight away by addi x3, x3, 8.

The `lw` computes its address in EX and reads memory in MEM, so the value of x3 exists only at the end of MEM. The `addi` wants x3 at the start of its own EX stage, which is one cycle too early. Forwarding cannot fix this, so the hazard unit holds the PC and the IF/ID register for one cycle and sends a NOP into EX. That NOP is the bubble.

The paper's observation is that the load address depends only on rs1 and the immediate. Both are often known while the load is still in ID, so the address can be computed there and the memory read moved into EX.

2.1 The same three instructions, cycle by cycle

The two tables below follow `lw x3, 0(x6)`, `addi x3, x3, 8` and `add x3, x3, x3` through the pipeline. They are taken from the paper's own Table I program, which is used again in section 6.2.

In the baseline, `addi` reaches ID in cycle 3 but cannot go on, because x3 does not exist yet. It sits in ID for a second cycle while a NOP goes down the pipeline in its place.

Instruction	C1	C2	C3	C4	C5	C6	C7	C8
lw x3, 0(x6)	IF	ID	EX	MEM	WB			
addi x3, x3, 8		IF	ID	ID (stall)	EX	MEM	WB	
add x3, x3, x3			IF	IF (stall)	ID	EX	MEM	WB

In the LW-driven core the address is ready at the end of the load's ID cycle, so memory is read in EX. The loaded value is already in the EX/MEM register when `addi` enters EX in cycle 4, and ordinary forwarding hands it over.

Instruction	C1	C2	C3	C4	C5	C6	C7
lw x3, 0(x6)	IF	ID, address computed	EX, memory read	MEM	WB		
addi x3, x3, 8		IF	ID	EX	MEM	WB	
add x3, x3, x3			IF	ID	EX	MEM	WB

The last instruction finishes in cycle 7 instead of cycle 8. One load, one cycle saved; across a program with many loads those cycles add up.

3. Design

3.1 Baseline core

The baseline is a standard RV32I five-stage pipeline with pipeline registers between every stage, a forwarding unit (EX/MEM and MEM/WB to EX), and a hazard unit. The hazard unit stalls for one cycle whenever the instruction in EX is a load and the instruction in ID reads its destination register. Branches resolve in MEM and flush the younger instructions.

3.2 LW-driven bubble-free core

The second core keeps everything above and adds four things, following the paper:

- An address generator in ID that adds rs1 and the immediate while the load is still being decoded.
- A bubble mitigation control unit that decides where rs1 comes from and whether a stall is still needed.
- A second read port on the data memory, which reads at the early address during the load's EX stage.
- A small mux in EX that puts the early-read data into the EX/MEM register in place of the ALU result, so the next instruction picks it up through the normal EX/MEM forwarding path.

The control unit sorts every load into one of three cases, depending on where its base register rs1 is produced.

Case	Where rs1 comes from	What the control unit does	Stall
Case 1	Already in the register file	AG uses the register file value directly	None
Case 2	Instruction two ahead, now in EX/MEM	Forwards the EX/MEM result into the AG	None
Case 3	Instruction directly ahead, still in EX	Value not ready in time, so it holds the load for one cycle; it then proceeds as Case 2	1 cycle

Case 3 is the only one where a bubble remains, and the paper accepts this because the producer's result simply does not exist yet. In the paper's own experiments Case 3 is also the rarest pattern, 0.7 loads per experiment on average against 5.5 for Case 1.

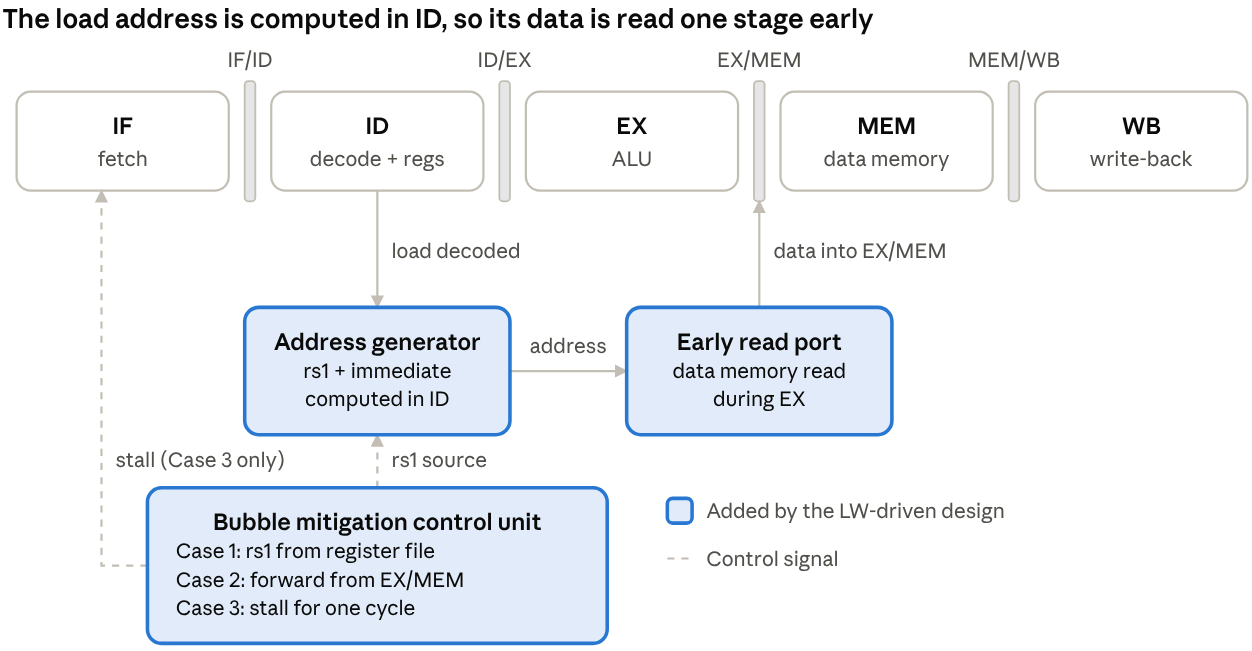


Figure 1. LW-driven pipeline, additions in colour

The address generator and the early read port take the load's memory access out of MEM; the control unit chooses where rs1 comes from and stalls only for Case 3.

4. Implementation

Both cores start from the same open-source RV32I pipeline (saravana-kumar-T-J-A/32-bit-Risc-V-processor-core). I kept two separate Vivado projects so the only differences between them are the bubble mitigation changes.

- Tools: Vivado 2025.2 for simulation, synthesis and implementation; Icarus Verilog 12 to cross-check every simulation result.
- Board: EDGE Artix-7 kit, XC7A35T-FTG256, with a 50 MHz on-board oscillator; the constraint file asks for a stricter 10 ns clock.

4.1 Module hierarchy

In both projects `fpga_top` is the top module, and it contains the processor, `riscv_top`. Inside the processor there is one module for each pipeline stage, plus the register file, the two memories, and the hazard and forwarding units. The LW-driven project has one extra module, `bubble_ctrl`. The table below lists every module, what it does, and what changed in the LW-driven core.

Module (file)	Stage	What it does	Change in the LW-driven core
pc (pc.v)	IF	Program counter; holds its value on a stall	none
instr_mem (instr_mem.v)	IF	256-word instruction ROM	none
if_stage (if_stage.v)	IF	IF/ID pipeline register	none
register_file (reg_file.v)	ID, WB	32 registers of 32 bits, x0 always 0	none
id_stage (id_stage.v)	ID	Decoder, immediate, control signals, ID/EX register	gives opcode and immediate to the AG; carries the early address
bubble_ctrl (bubble_ctrl.v)	ID	Address generator and bubble mitigation control unit	new module
hazard_unit (hazard_unit.v)	ID	Stall and flush control	stalls only for Case 3

Module (file)	Stage	What it does	Change in the LW-driven core
ex_stage, alu (ex_stage.v, alu.v)	EX	Forwarding muxes, ALU, EX/MEM register	mux for the early-read data
forwarding_unit (forwarding_unit.v)	EX	EX/MEM and MEM/WB forwarding	none
mem_stage (mem_stage.v)	MEM	Data memory access, branches, MEM/WB register	none
data_mem (data_mem.v)	MEM	256-word data memory	second read port
wb_stage (wb_sage.v)	WB	Picks the ALU result or loaded word for write-back	none
fpga_top (fpga_top.v)	board	Cycle counter and LED display	used in both projects

Both projects also share two fixes: the register file write-through (section 9) and an instruction ROM that has the test program built in for synthesis.

4.2 The bubble mitigation control unit (bubble_ctrl.v)

`bubble_ctrl` is the heart of the project. It watches the instruction in ID, and when that instruction is a load it makes its decisions in the same cycle, before the load moves on.

First it checks whether the load's base register rs1 is about to be written by the instruction directly ahead, which is now in EX. If it is, the value does not exist yet. This is Case 3: the unit raises a stall request and the hazard unit holds the load in ID for one cycle.

If rs1 is instead being written by the instruction two places ahead, that result is already sitting in the EX/MEM register. The unit takes the value from there rather than from the register file. This is Case 2.

In every other situation rs1 comes straight from the register file, which is Case 1.

For Case 1 and Case 2 the unit adds rs1 and the immediate, which is the address generator, and passes the address and a valid flag into the ID/EX register. So when the load reaches EX in the next cycle, its address is already known.

4.3 Changes to the existing modules

The rest of the technique is a few small edits.

The data memory got a second read port. It reads the word at the early address during the load's EX stage, separately from the normal port used in MEM.

In the EX stage, a two-way mux then puts this early-read word into the EX/MEM register in place of the ALU result. For a load the ALU would only have produced the address, which is no longer needed, so nothing is lost. From there the next instruction picks up the value through the existing EX/MEM forwarding path, exactly as it would for an ALU result.

In `id_stage.v`, an early load is set to write back through the ALU path, since its data now sits where the ALU result normally goes. In `hazard_unit.v`, the old load-use check is gone: the unit stalls for a load only when `bubble_ctrl` reports Case 3.

4.4 The FPGA wrapper (fpga_top.v)

`fpga_top` is the module Vivado synthesizes. It wraps the processor with a counter that runs from the release of the reset button until the PC reaches the end marker at 0x80, latches register x30 six cycles later, and shows either value on the LEDs depending on switch sw0. LED 15 lights when the result is ready. Two debug outputs, `dbg_pc` and `dbg_x30`, were added to `riscv_top` for this; they do not change how the processor behaves.

5. Verification method

The same program always runs on both cores, and the correctness check is that both end with identical register values. Only the cycle count is allowed to differ.

Three kinds of test program were used:

1. The paper's Table I program: seven instructions with one load-use hazard, machine code copied exactly as printed in the paper. I used it for the waveform comparison.
2. A 10-load program of 33 instructions. Every load is followed by an instruction that needs its value, and the loads cover all three cases.
3. A Table III-style sweep: 45 programs made by a Python script from 5 hazard mixes, each repeated 1, 2 and 3 times, for three techniques (stall, compiler reordering and bubble mitigation). The loops are unrolled instead of written with branches, because the base code's branches are broken (section 9). An unrolled loop runs exactly the same instructions, so the cycle count is the same as a working loop would give.

The testbenches count cycles from reset until the PC reaches a marker instruction at the end of the program, then wait for the pipeline to drain before reading registers. Separate testbenches log every bubble (baseline) or every hazard case (LW-driven) cycle by cycle, and expose paper-style waveform signals: `BUBBLE_DETECTED`, `BUBBLE_INSERTED` and `BUBBLE_FIX`.

The paper's assembly column writes the fourth and seventh instructions as `add R3, R3, R8` and `add R3, R9, R8`, but the machine code it prints is `add x3, x3, x3` in both places. I ran the machine code as printed. The expected write-back sequence is 4, 2, 8, 4, 12, 24.

5.1 The 10-load program

The first six instructions store 111 at address 0 and 222 at address 4. After that come ten load-and-use pairs, arranged so the control unit sees all three cases. The last column is the case the LW-driven core reported for each load.

Address	Instructions	What they do	Load case
0x00 to 0x14	addi x1, x0, 111; addi x2, x0, 0; sw x1, 0(x2); addi x3, x0, 222; addi x4, x0, 4; sw x3, 0(x4)	store 111 at address 0 and 222 at address 4	
0x18	addi x28, x0, 0	base register x28 = 0	
0x1c, 0x20	lw x29, 0(x28); addi x30, x29, 1	load and use	Case 3 (x28 set just before)
0x24 to 0x50	the same lw and addi pair, six times	load and use	Case 1, six times
0x54, 0x58	addi x28, x0, 4; addi x13, x0, 5	base = 4, then a filler	
0x5c, 0x60	lw x29, 0(x28); addi x30, x29, 1	load and use	Case 2 (x28 set two instructions ahead)
0x64 to 0x70	the same four instructions again		Case 2
0x74	addi x28, x0, 0	base = 0 again	

Address	Instructions	What they do	Load case
0x78, 0x7c	lw x29, 0(x28); addi x30, x29, 1	load and use	Case 3
0x80	jal x31, 0	end-of-program marker	

When it finishes, x28 should be 0, x29 should be 111 (the last load reads address 0) and x30 should be 112. The `jal` at 0x80 is only a marker: the testbenches and the FPGA wrapper stop counting when the PC reaches it, so it does not matter that the base code cannot execute `jal`.

6. Simulation results

6.1 Bubble detection on the 10-load program

The bubble-free core removed 8 of the 10 bubbles and finished 7 cycles earlier, with the same final state.

	Baseline	LW-driven
Loads executed	10	10
Hazard cases detected	not classified	Case 1: 6, Case 2: 2, Case 3: 2
Bubbles inserted	10	2 (both Case 3)
Cycles to complete	41	34
Final x28, x29, x30	0, 111, 112	0, 111, 112

Eight bubbles were removed but the gap is seven cycles. The baseline's last bubble happens after the final instruction has already been fetched, so it does not delay the point where the cycle count stops.

```
Time resolution is 1 ps
===============================================================
 BASELINE (conventional stall) -- live hazard detection log
===============================================================
CYCLE | PC in ID | EVENT
------+----------+-----------------------------------------------
    9  | 0x20     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   12  | 0x28     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   15  | 0x30     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   18  | 0x38     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   21  | 0x40     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   24  | 0x48     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   27  | 0x50     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   32  | 0x60     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   37  | 0x70     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
   41  | 0x7c     | LOAD-USE HAZARD -> BUBBLE INSERTED (1 cycle stall)
------+----------+-----------------------------------------------

SUMMARY: 10 loads executed | 10 bubbles (stall cycles) inserted

TOTAL_CYCLES_TO_COMPLETE = 41
FINAL STATE: x28=0 x29=111 x30=112
(must be identical to the LW-driven log -> same answer, different cycles)
$finish called at time : 495 ns : File "C:/Vivado_projects/RiscV_32_pipelined/RiscV_32_pipelined.srcs/sim_1/new/riscv_tb.v" Line 70
```

Figure 2. Baseline, Vivado console: every one of the 10 load-use hazards inserts a bubble; 41 cycles.

```
Time resolution is 1 ps
===============================================================
 LW-DRIVEN BUBBLE MITIGATION -- live hazard-case detection log
===============================================================
CYCLE | PC in ID | CASE DETECTED  | RESULT
------+----------+----------------+-------------------------------
    8  | 0x1c     | Case 3         | BUBBLE INSERTED (1 cycle stall)
    9  | 0x1c     |   (continued)  | released, address via EX/MEM forward
   11  | 0x24     | Case 1         | bubble MITIGATED (early address)
   13  | 0x2c     | Case 1         | bubble MITIGATED (early address)
   15  | 0x34     | Case 1         | bubble MITIGATED (early address)
   17  | 0x3c     | Case 1         | bubble MITIGATED (early address)
   19  | 0x44     | Case 1         | bubble MITIGATED (early address)
   21  | 0x4c     | Case 1         | bubble MITIGATED (early address)
   25  | 0x5c     | Case 2         | bubble MITIGATED (EX/MEM fwd to AG)
   29  | 0x6c     | Case 2         | bubble MITIGATED (EX/MEM fwd to AG)
   32  | 0x78     | Case 3         | BUBBLE INSERTED (1 cycle stall)
   33  | 0x78     |   (continued)  | released, address via EX/MEM forward
------+----------+----------------+-------------------------------

SUMMARY: 10 loads executed | Case1=6 Case2=2 Case3=2
  -> 8 of 10 load-use bubbles MITIGATED (Case 1 + Case 2)
  -> 2 bubbles (stall cycles) still inserted (Case 3, as the paper specifies)

TOTAL_CYCLES_TO_COMPLETE = 34
FINAL STATE: x28=0 x29=111 x30=112
(must be identical to the baseline log -> same answer, different cycles)
$finish called at time : 425 ns : File "C:/Vivado_projects/LW_driven_Riscv/LW_driven_Riscv.srcs/sim_1/new/riscv_tb.v" Line 89
```

Figure 3. LW-driven, Vivado console: each load is classified as Case 1, 2 or 3; only the two Case 3 loads stall; 34 cycles.

6.2 The paper's Table I program

Both cores detect the load-use hazard (`lw x3` followed by `addi x3, x3, 8`) in cycle 6. The baseline answers it with a bubble and the PC stays at 0x18 for two cycles. The LW-driven core already has the data, so `BUBBLE_FIX` pulses, `BUBBLE_INSERTED` stays low and the PC keeps moving.

The write-back sequence 4, 2, 8, 4, 12 (0x0c), 24 matches the output shown in the paper's waveform figure, and the bubble-free core finishes one cycle earlier.

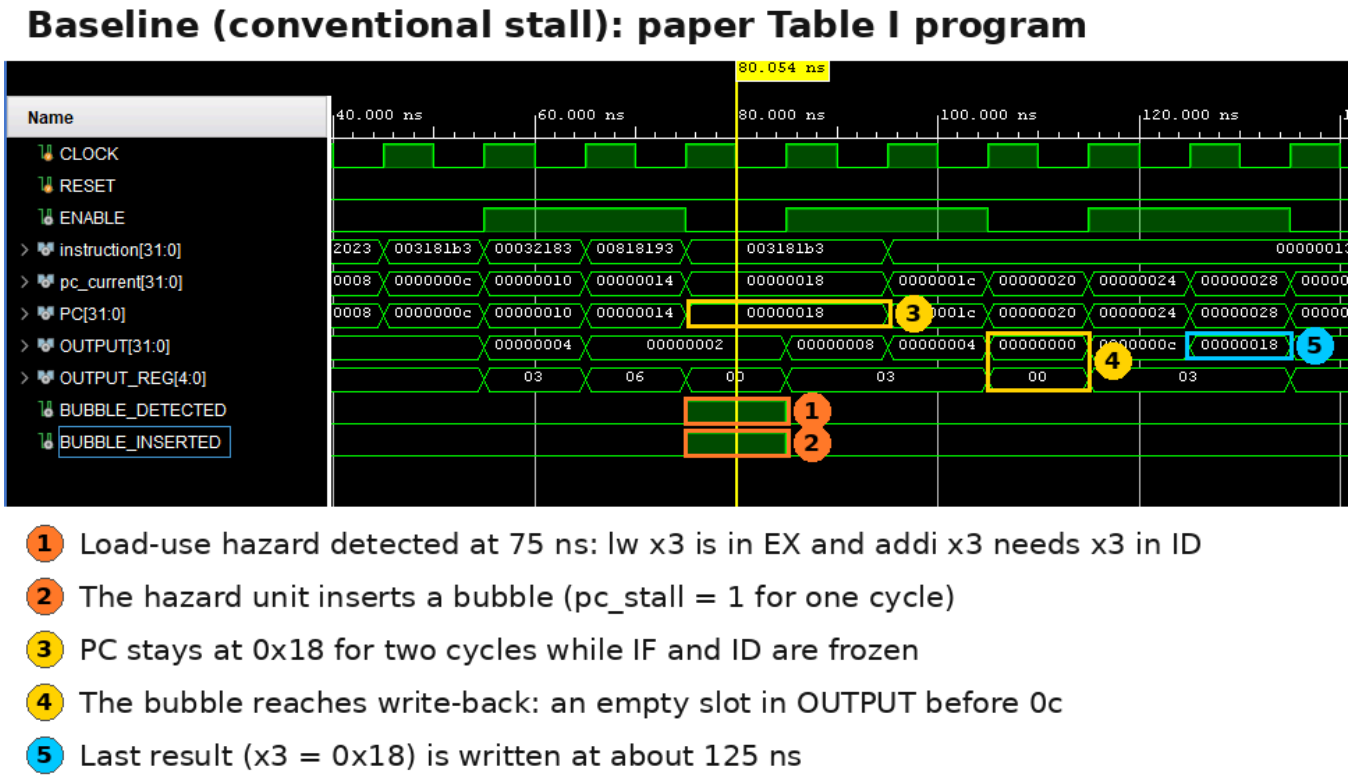


Figure 4. Baseline waveform, 40 to 140 ns. The hazard is detected at 75 ns (1), a bubble is inserted (2), the PC holds 0x18 for two cycles (3), the bubble shows up as an empty write-back slot (4), and the last result arrives at about 125 ns (5).

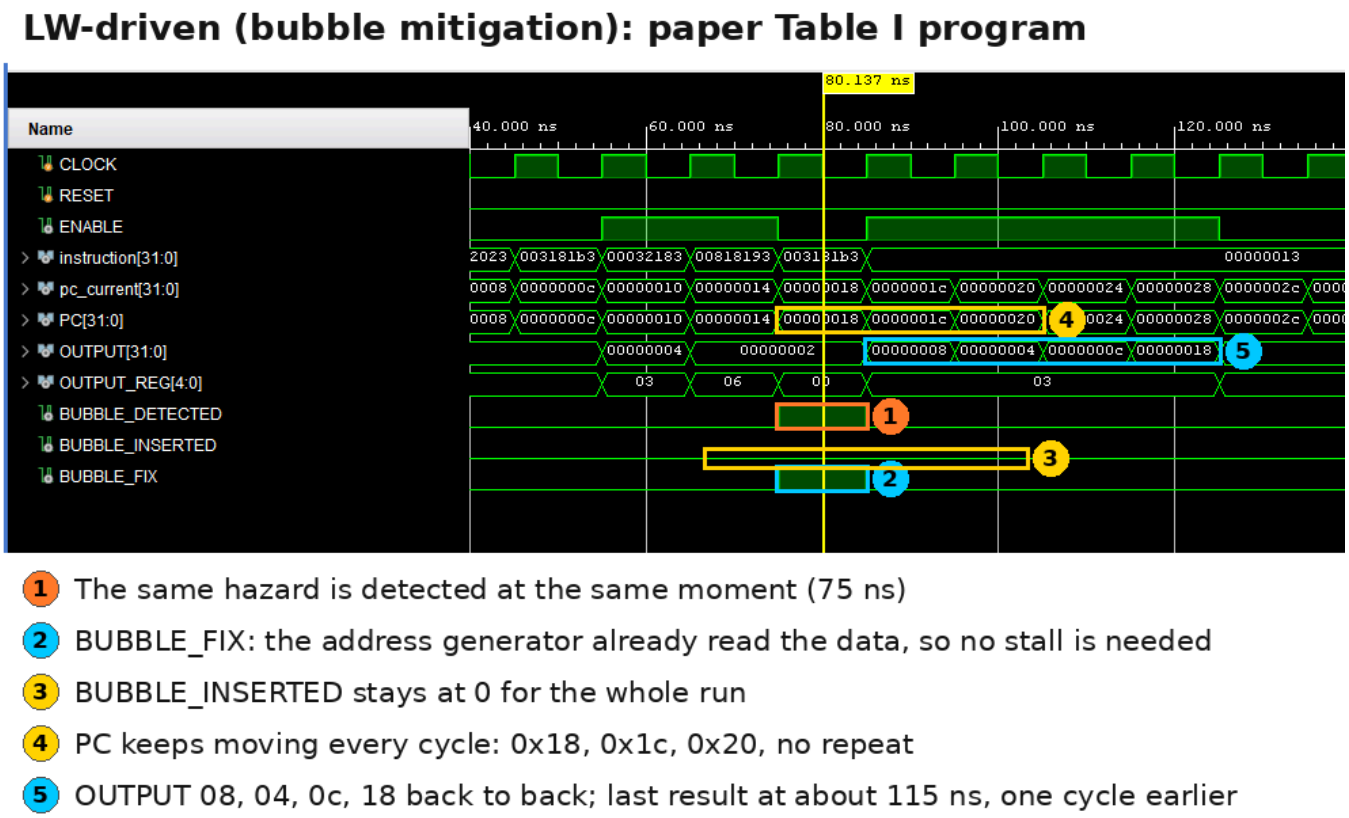


Figure 5. LW-driven waveform, same window. The same hazard is detected at 75 ns (1) but BUBBLE_FIX fires instead of a stall (2), BUBBLE_INSERTED never rises (3), the PC never repeats (4), and the results arrive back to back, the last one at about 115 ns (5).

6.3 Table III-style sweep

Five hazard mixes, from five Case 1 loads (Ex-1) to a mix of all three cases (Ex-3), were each run with 1, 2 and 3 repetitions, giving 15 configurations per technique.

Technique	Total cycles, all 15 configurations
Conventional stall	591
Compiler reordering	606
LW-driven bubble mitigation	483

Bubble mitigation used 18.27% fewer cycles than stalling, and every configuration produced identical final register values across all three techniques. The improvement grows with the number of repetitions, since the fixed pipeline fill time becomes a smaller share of the total (figure below).

My compiler reordering column inserts an independent instruction after each load instead of moving existing code, so it adds instructions and comes out slower than stalling. It is not a fair copy of the paper's method.

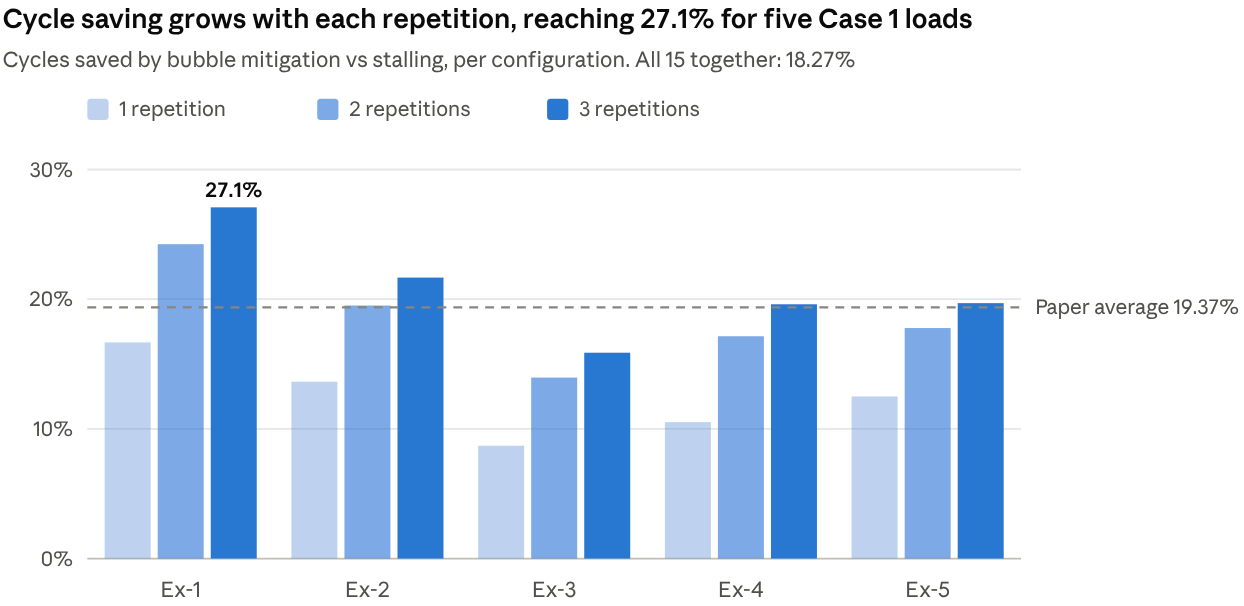


Figure 6. Table III-style sweep, cycle counts from simulation (15 configurations)

The two mixes that include a Case 3 load, Ex-3 and Ex-4, gain the least, because that load still stalls.

7. FPGA results

Both cores ran the 10-load program on the Artix-7 board and gave the same answer, with the bubble-free core finishing in 34 cycles against 41.

The hardware numbers match simulation exactly. The same x30 pattern on both bitstreams confirms the result is correct, and the cycle count confirms the 17.1% saving on real silicon.

7.1 Pin assignment

The constraint file `fpga_top.xdc` uses the pin list from the EDGE Artix-7 kit's documentation.

Port of fpga_top	Direction	Part on the board	FPGA pin
clk	in	on-board oscillator	N11
btn_rst	in	centre push button SW22 (pull-down, so pressed = 1)	M14
sw0	in	rightmost slide switch	L5
led[15:0]	out	the row of 16 LEDs above the switches	see the next table

The LEDs show a binary number. To read one, add up the weights of the LEDs that are on: for the baseline, LEDs 5, 3 and 0 give 32 + 8 + 1 = 41.

LED	FPGA pin	Weight	LED	FPGA pin	Weight
0	J3	1	8	K5	256
1	H3	2	9	P6	512
2	J1	4	10	R7	1024
3	K1	8	11	R6	2048
4	L3	16	12	T5	4096
5	L2	32	13	R5	8192
6	K3	64	14	T10	16384
7	K2	128	15	T9	done flag

One thing I only noticed while photographing the board: the oscillator next to pin N11 is marked 50.000 MHz, not 100 MHz. The constraint file asks for a 10 ns clock, which is twice as strict as the real 20 ns, so both designs have even more timing margin on the board than section 8 shows. The cycle counts do not depend on the clock frequency at all.

7.2 What the board showed

In each photo below, the LED numbers are written above the LEDs, the lit ones are circled in yellow, and the reset button, switch sw0 and the FPGA's DONE LED are marked in blue. The DONE LED is the FPGA's own configuration indicator: it lights once the bitstream has loaded.

Baseline core, sw0 = 0 (cycle count)

LEDs 5, 3, 0 ON -> 101001 (binary) = 41 cycles LED 15 = done

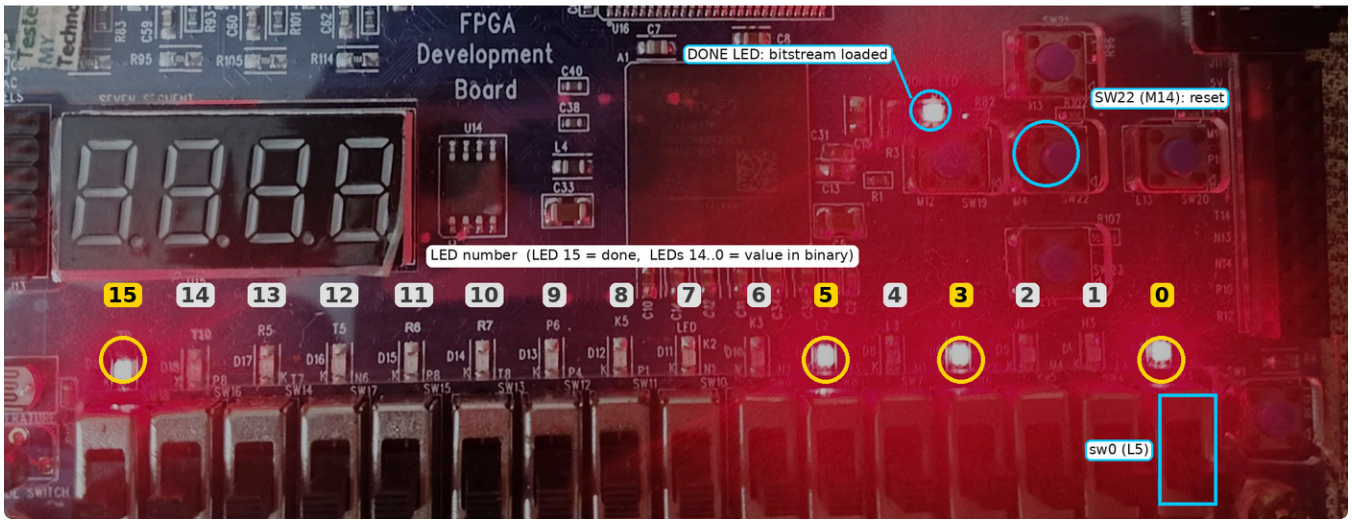


Figure 7. Baseline bitstream with sw0 = 0. LEDs 5, 3 and 0 are on: 101001 in binary, 41 cycles.

LW-driven core, sw0 = 0 (cycle count)

LEDs 5, 1 ON -> 100010 (binary) = 34 cycles LED 15 = done

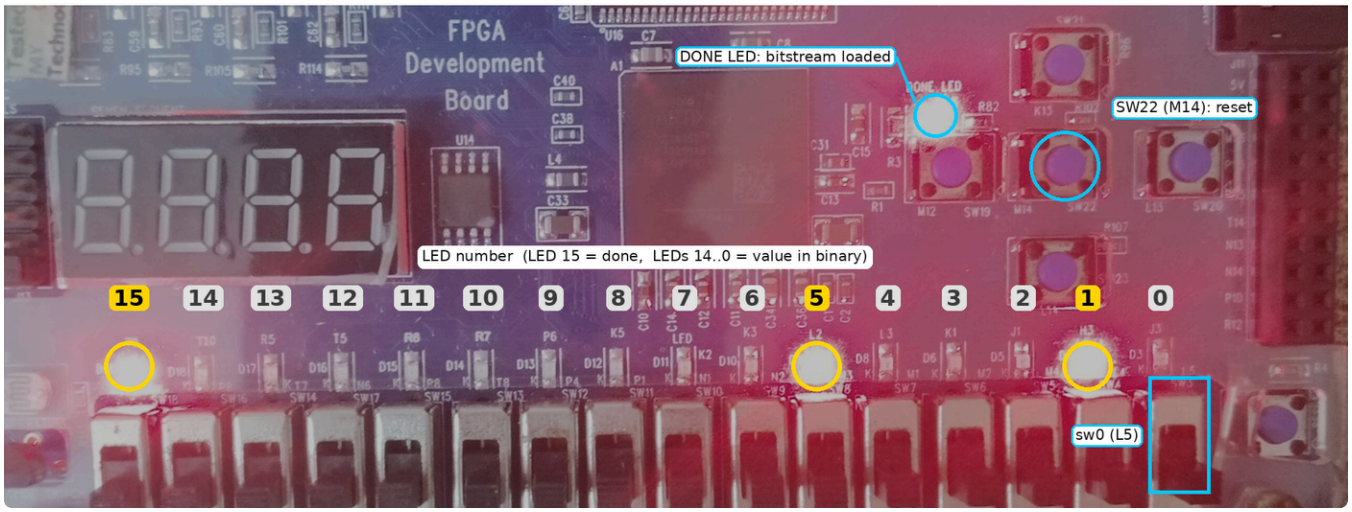


Figure 8. LW-driven bitstream with sw0 = 0. LEDs 5 and 1 are on: 100010 in binary, 34 cycles.

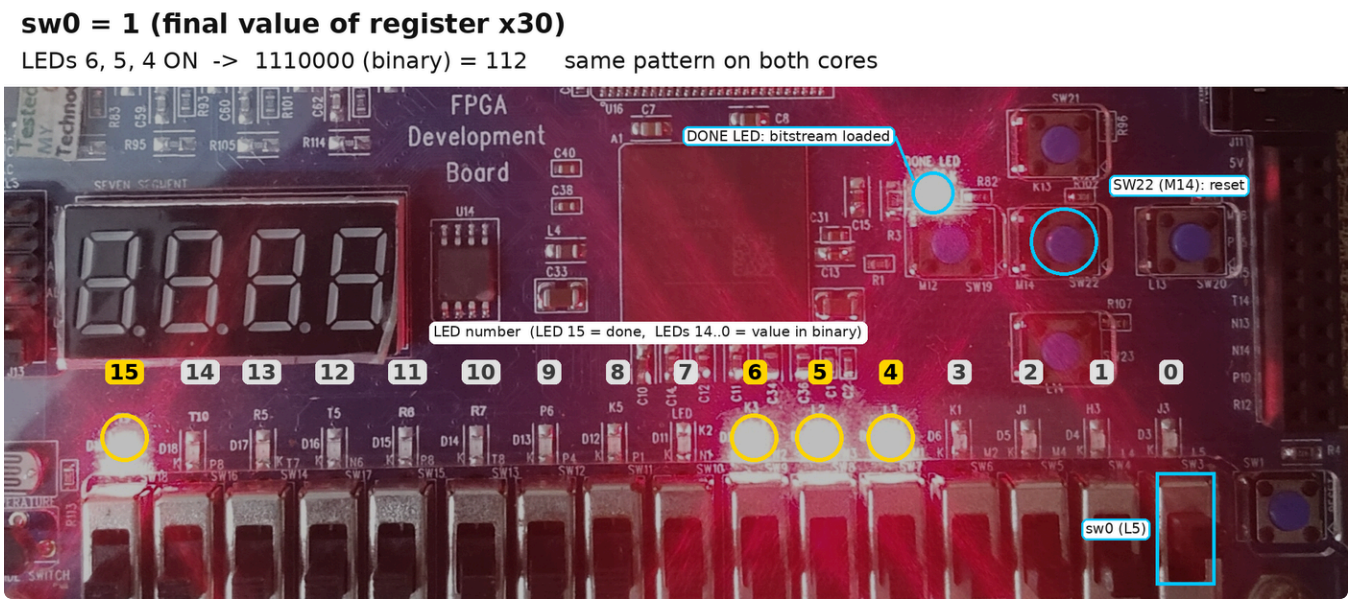


Figure 9. sw0 = 1 shows the final value of x30. LEDs 6, 5 and 4 are on: 1110000 in binary, 112. Both bitstreams show this same pattern.

8. Area, timing and power

The bubble-free core needs 12.2% more LUTs and registers and has a 5.2% lower maximum frequency, but its estimated dynamic power is 3 mW lower. This is the same trade-off the paper reports in its Table II.

All numbers below come from the routed designs in Vivado 2025.2 (`fpga_top` as top, xc7a35tftg256 at speed grade -3, 10 ns clock constraint).

Parameter	Baseline	LW-driven	Change
Slice LUTs as logic	796	772	-3.0%
Slice LUTs as distributed RAM	0	256	+256
Block RAM (RAMB18)	1	0	-1
Slice registers	1280	1302	+1.7%
LUTs + registers	2076	2330	+12.2%
Worst negative slack at 10 ns (ns)	3.782	3.441	-0.341
Max delay = 10 - WNS (ns)	6.218	6.559	+5.5%
Max frequency (MHz)	160.8	152.5	-5.2%

Parameter	Baseline	LW-driven	Change
Dynamic power (mW)	22	19	-13.6%
Dynamic power, processor only (mW)	11	8	-27.3%
Total on-chip power (mW)	93	91	-2.2%
Dynamic mW/MHz (at 100 MHz)	0.22	0.19	-13.6%
Dynamic energy for the 10-load program (nJ)	9.0	6.5	-28.4%

Most of the extra area is memory, not logic. Leave the memory out and the LUT plus register count hardly moves (2076 against 2074). What changed is where the data memory lives. With one read port, Vivado packed the 1 KB data memory into a block RAM. The early load needs a second, independent read port, and that pushed the memory into 256 LUTs of distributed RAM instead.

The power figures should be read with some care. They come from Vivado's vectorless estimate, which it rates as medium confidence, and they are rounded to 1 mW, so the 3 mW drop is approximate. Part of it comes from the memory moving out of the block RAM, which drew 2 mW in the baseline. The estimate also assumes the 100 MHz clock in the constraint file. At the board's 50 MHz, the power would be roughly halved, but the energy per cycle, and therefore the energy for a whole program, stays about the same.

The energy row is where the technique really pays off. Energy for a program is power multiplied by run time. The bubble-free core runs the 10-load program in 34 cycles instead of 41, at slightly lower power, so its dynamic energy is about 28% lower.

Comparison with the paper's Table II

Parameter	Paper: stalling core	Paper: proposed core	Paper change	Change in this project
LUTs + registers	1716	1912	+11.4%	+12.2%
Max delay (ns)	7.684	7.931	+3.2%	+5.5%
Max frequency (MHz)	130	126	-3.0%	-5.2%
Power (mW)	121	114	-5.8%	-13.6% (dynamic)
mW/MHz	0.93	0.90	-3.2%	-13.6% (dynamic)

The area cost is very close to the paper's. My timing penalty is a little larger and my power saving is larger. The raw numbers differ from the paper's because the device, speed grade and design details differ: here the register file alone takes about 1000 flip-flops, and the core is synthesized for one small fixed program, so Vivado removes logic that program never uses. The percentage changes are the meaningful comparison, not the absolute values.

9. Problems found and how they were handled

Four bugs were already present in the base code. None of them comes from the bubble mitigation logic, and each one affects both cores equally.

Problem	Symptom	How it was handled
Branches ignore funct3	Every branch (bne, blt, ...) behaves like beq	Test loops unrolled, no branches needed
No jal decoding	Unconditional jumps are silently skipped	End of program detected from the PC instead
Branch resolves in MEM, but only two slots are flushed	One wrong-path instruction executes after a taken branch	Test loops unrolled

Problem	Symptom	How it was handled
Register file has no write-through	A register read in ID in the same cycle it is written in WB returns the old value; `add x3, x3, x3` gave 0 instead of 8	Fixed with a two-line bypass in reg_file.v, applied to both cores

After the register file fix, the full 45-program sweep was run again. Every cycle count stayed the same and all results still matched.

One more issue came up during the FPGA work.

The first bitstream I built ran an empty program. `$readmemh` with a relative file name works in simulation, but Vivado synthesis runs from a different folder, could not find `instructions.mem`, and quietly built an empty instruction ROM. The board showed 32 cycles and x30 = 0, which is exactly what a CPU stepping through blank memory would do. The fix was to write the test program into `instr_mem.v` directly for synthesis, and keep `$readmemh` for simulation.

10. Comparison with the paper

The cycle saving I measured is close to the paper's, even though the test programs are smaller and different.

Metric	Paper (Kumar and Ray)	This project
Average cycle improvement over stalling	19.37% over 10 experiments	18.27% over 15 configurations; 17.1% on the 10-load program
Improvement range	4.67% to 22.91%	8.70% to 27.08%
Hazard cases	Case 1 and 2 without stall; Case 3 with a 1-cycle stall	Same
Area, Fmax, power	+11.4%, -3.0%, -5.8%	+12.2%, -5.2%, -13.6% (section 8)
Hardware check	FPGA implementation	Artix-7 XC7A35T, both cores, same result

The paper also reports CoreMark CPI (1.36 to 1.21) and a comparison with other low-power cores (SCR1, PicoRV32, CV32E40P, E21, N22). Those were not reproduced here, because running CoreMark needs working branches and jumps, which the base code does not have.

11. Conclusion and future work

The LW-driven bubble mitigation scheme works as the paper describes. Moving the load address calculation into ID removed every Case 1 and Case 2 bubble, left only the Case 3 stall, and never changed a result. The saving was 17 to 18% in simulation and the same 41-to-34-cycle reduction was seen on a real FPGA.

The price is mostly in memory. The second read port moved the data memory from a block RAM into 256 LUTs, which accounts for nearly all of the 12.2% area increase, and the maximum frequency dropped by 5.2%. Estimated dynamic power fell from 22 to 19 mW, and because the program also finishes sooner, the dynamic energy for the 10-load program dropped by about 28%. For an energy-constrained core, which is the paper's target, that is the number that matters most.

Next steps:

- Fix the branch and jal bugs so real loops and CoreMark can run, and compare CPI with the paper.
- Make the instruction memory loadable (for example over UART) so area and power are measured on a full core rather than one trimmed for a single program.
- Measure power on the board instead of relying on Vivado's estimate.

References

1. Kumar and Ray, "Micro-Architecture of LW Driven Bubble-Free Five-Stage Pipelined RISC-V Processor Core for Energy Constraint Low-End Application," IEEE Transactions on Circuits and Systems I: Regular Papers, January 2026.
2. saravana-kumar-T-J-A, 32-bit RISC-V processor core (base RTL), GitHub.
3. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, RISC-V International.
4. D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition, Morgan Kaufmann.